ANN Cheatsheet

① Perceptron wieght updation :

$$w_{new} = w_{old} + \eta * Error * x$$

$$Error = y_{actual} - y_{Predicted}$$

② Back propogation:

weight updation: $w_{i new} = w_{i old} + \eta \frac{dL}{dw_{i old}}$

Chain rule is used

for diamond like node

we use addition along with chain rule

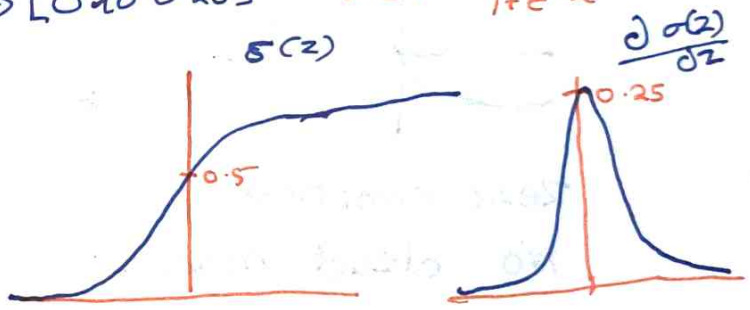
③ Activation function: [for generating non linearity]

(a) Sigmoid Activation function → [0, to 1]

$\frac{d\,\sigma(z)}{dz} \rightarrow [0 \text{ to } 0.25]$ $\quad \sigma(z) = \frac{1}{1+e^{-x}}$

deminishig gradient

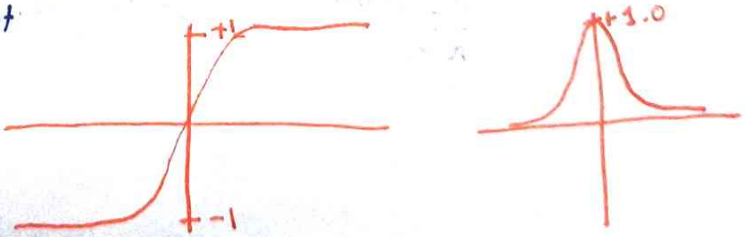
not zero centered



(b) Tanh activation function

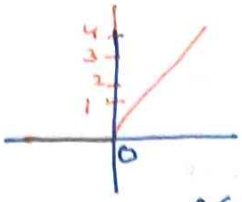
$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Still vanishig gradient

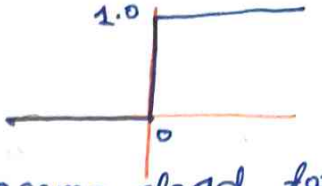


(c) Relu Activation function

$\max(0, x)$



$\frac{\partial \text{Relu}(x)}{\partial x}$



Neuron can become dead for some eg

→

(d) Leaky Relu Parametric Relu

↓

$\max(\alpha x, x)$

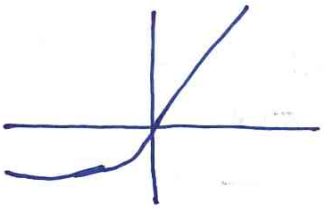
α → Small value

α → learnt → parametric Relu
α → fixed → leaky Relu

Removes dead neuron

(e) Elu

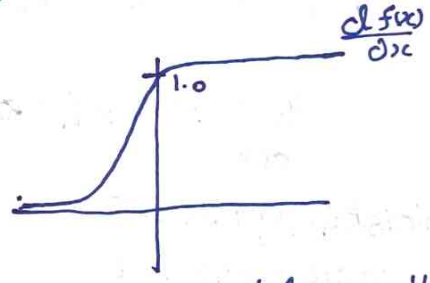
$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha(e^x - 1) & \text{if otherwise} \end{cases}$$



Zero centered

No dead neuron

$\frac{\partial f(x)}{\partial x}$



complex maths

(f) Softmax

$$\frac{e^{y_i}}{\sum_{K=0}^{n} e^{y_K}}$$

→ multi-class classification

→ ouput layer

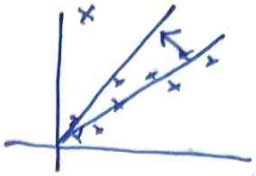
(4) Loss functions / cost functions (Regression)

(a) **Mse** $(y-\hat{y})^2$ → loss fucn

$\frac{1}{n}(y-\hat{y})^2$ → cost fucn

→ Differentiable
→ only 1 minima
→ converges faster

→ Not Robust to Outliers



(b) **MAE** $= |y-\hat{y}|$ → loss

$\frac{1}{n}|y-\hat{y}|$ → cost

Robust to outlier

convergence takes time

(c) **Huber loss**

$$\text{Cost fucn} = \begin{cases} \frac{1}{n}\sum(y-\hat{y})^2 & , \text{ if } |y-\hat{y}| \le \delta \\ \delta|y-\hat{y}| - \frac{1}{2}\delta^2 & , \text{ otherwise} \end{cases}$$

→ Hyper-parameter

(d) **RMSE**

$$\text{Cost fucn} = \sqrt{\frac{\sum_{i=1}^{N}(y_i-\hat{y}_i)^2}{N}}$$

Loss or cost functions (Classification)

(a) Binary cross entropy

$$-y(\log(\hat{y})) - (1-y)(\log(1-\hat{y}))$$

↳ activation for this is sigmoid

(b) Categorical cross entropy

$$-\sum_{j=1}^{c} y_{ij} * \ln(\hat{y}_{ij})$$

Softmax activation

(c) Sparse categorical cross entropy

We take index → of probability which corresponds to value 1 in actual

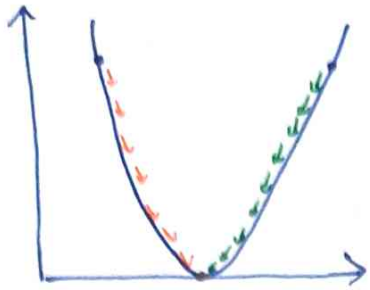
eg: [0,0,1,0] → Actual [0.1, 0.5, 0.3, 0.1] → Prediction

loss = $-\log(0.3)$

⑤ Optimizers

(a) Gradient Descent

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w_{old}}$$



iteration : one forward pass + backward pass + weight update using one batch

Epoch : One complete pass through entire dataset

* only one update happens after entire dataset is parsed
  ↳ There can be mutiple epochs
  ↳ More Ram

(b) Stochastic Gradient descent (SGD)

for every data point there is backward propogation & weight updation
There can be multiple epoch

- Solves Ram problem

* Time complexity
* Convergence will take time
* Noise is introduced

(c) Mini Batch SGD

1000 data point
↓
100 batch size → 10 batch

10 iteration → forward → backward → weight updat
→ every iteration

Can have multiple epoch
less noise
fast convergence
Efficient Ram

(d) SGD with momentum

SGD work with current gradient but SGD with momentum remembers the past update like velocity and it keeps moving in same direction

$\beta \rightarrow$ momentum cofficient

$v_t = \beta v_{t-1} - \eta g_t$ (velocity update)

$\theta_t = \theta_t + v_t$ (parameter update)

$0 < \beta < 1$ usually 0.9

Normal SGD

iteration	Gradient	$-\eta g$	weight
Initial	–	–	3.000
1	-6	0.6	3.6
2	-6	0.6	4.2
3	-6	0.6	4.8
4	+2	-0.2	4.6
5	-6	+0.6	5.2

SGD with moment

iteration	Gradient	v_t	weight
Initial	–	0	3.0
1	-6	0.6	3.6
2	-6	1.14	4.74
3	-6	1.626	6.366
4	+2	1.263	7.6294
5	-6	1.737	9.3665

(e) Adagard

$$g_t = \frac{\partial L}{\partial w}$$

Initial $G_t = 0$

$$G_t = G_{t-1} + g_t^2$$

$$\eta' = \frac{\eta}{\sqrt{G_t + \varepsilon}}$$

Calculate gradient → Calculate learning rate → Update weight

RMSPROP (Root mean square propogation)

Idea: Why should gradient from 1000 iterations ago still affect today's learning rate

Adagrad

Iteration 1

Iteration 2

Iteration 3

.....

Iteration 10000

Never forgets anything

RMSPROP

old gradient ✗ forget gradually

Recent gradient ✓ keep

$$S_t = \beta S_{t-1} + (1-\beta) g_t^2$$

$\beta \approx 0.9$

$\varepsilon \rightarrow$ small number

g_t = current gradient

S_t = moving average square gradient

Old history 90% $\rightarrow \beta$

Current gradient $\rightarrow$ 10%

Adams Optimizer

Adam stores weight, momentum, variance

Step 1 → compute gradient

Say $g = -4$

Step 2 → update momentum

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$

usually $\beta_1 = 0.9$

90% → old momentum 10% current gradient

Step 3 → Update Variance

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g t^2$$

β_2 usally 0.999

99.9 % old variance 0.1 % current variance

Why Bais correction

if we start with

$m = 0$ & $v = 0$

they are baised towards zero

eg : gradient = 10

momentum = $0.9(0) + (0.1)\,10 = 1$

But actual gradient = 10

momentum 1 → too small

Adam fixes it using Bais correction

$$\hat{m} = \frac{m_t}{1 - \beta_1^t} \qquad \hat{v} = \frac{v_t}{1 - \beta_2^t}$$

weight update = $W - \eta \frac{\hat{m}}{\sqrt{\hat{v} + \varepsilon}}$